

AI465 — Agentic AI Lab

Experiment 4: Retrieval-Augmented Generation — Grounding an Agent in a Document Store

Aim

Build a local RAG pipeline: chunk a corpus, vectorise it with TF-IDF, retrieve nearest chunks using cosine similarity, assemble grounded context, and compare grounded answers with an ungrounded baseline.

Theory

Retrieval-Augmented Generation moves knowledge from model weights into an external document store. Documents are split into chunks, converted into vectors, searched at query time, and supplied as evidence. Each chunk retains its source identifier, making answers traceable. This notebook uses TF-IDF and cosine similarity so the complete retrieval pipeline works without an API key or network access.

Stage 1 — Import Required Libraries

NumPy and Pandas handle data, while scikit-learn provides TF-IDF vectorisation and cosine similarity.

```
import re
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
print("Libraries imported successfully.")
```

Libraries imported successfully.

Stage 2 — Create the Document Corpus

Each document has a source identifier. The identifier is preserved through chunking and retrieval so answers can cite their evidence.

```
documents = {
    "RAG_Guide.txt": """
Retrieval-Augmented Generation, or RAG, combines information retrieval
with text generation. A RAG system retrieves relevant documents or
chunks before generating an answer. The retrieved context is supplied
to the language model as evidence. RAG can reduce hallucination by
grounding answers in an external document collection. The external
```

collection can be updated without retraining the language model. TF-IDF is a traditional technique for representing text as numerical vectors. TF-IDF gives importance to words that occur frequently in a document but less frequently across the complete corpus. Stop words can be removed during TF-IDF vectorisation.

Cosine similarity measures the angle between two vectors. A higher cosine similarity means that two vectors are more similar. In this experiment cosine similarity is used to rank document chunks for a query.

Chunking divides large documents into smaller pieces. Overlapping chunks allow information near a chunk boundary to appear in more than one chunk. Chunk size and overlap affect what information can be retrieved.

A vector store contains vector representations of documents or chunks. At query time, the query is converted into a vector and compared against stored vectors. This experiment performs exhaustive similarity search over the complete chunk matrix.

A similarity threshold can be used to prevent the system from answering when no retrieved chunk is sufficiently similar to the query. Without a threshold, top-k retrieval always returns some chunks, even when the corpus does not contain the requested information.

Citations make retrieved evidence traceable. Each chunk keeps its source document and position so that a generated answer can identify where its evidence came from.

Dense sentence embeddings can replace TF-IDF vectors in a more advanced RAG system. A FAISS index can also replace exhaustive search for large collections. The chunking and citation logic can remain unchanged.

""",

"AI_Notes.txt": """

Artificial intelligence systems can use external knowledge bases to improve factual responses. A knowledge base may contain technical documents, manuals, policies, or frequently updated information. Information retrieval ranks pieces of information according to their relevance to a user question. A retrieval system should return evidence that directly supports an answer.

A grounded answer should be based only on the supplied evidence. An ungrounded language model may provide fluent information without identifying its source. Therefore, citations improve the traceability and auditability of generated answers.

Retrieval quality depends on document preprocessing. Poor chunk boundaries can separate related information and make retrieval less effective. Using overlapping chunks can reduce this problem.

Increasing k causes more chunks to be retrieved. This can increase the amount of evidence available to the generator, but irrelevant chunks may also enter the context.

""",

"ML_Notes.txt": """

Machine learning models learn patterns from data. Classification predicts a category for an input while regression predicts a numerical value.

Vector representations are commonly used in machine learning. Text can be represented using sparse vectors such as TF-IDF or dense vectors produced by neural networks.

Similarity search is useful when a query needs to find related examples. Cosine similarity is commonly used for comparing text vectors.

A retrieval system should be evaluated separately from the generation system. Useful retrieval measurements include similarity scores, ranking, and whether the retrieved chunk contains the required evidence.

```
"""
```

```
}
```

```
print("Number of documents:", len(documents))
for source, text in documents.items():
    print(f"{source}: {len(text.split())} words")
```

```
Number of documents: 3
RAG_Guide.txt: 292 words
AI_Notes.txt: 135 words
ML_Notes.txt: 93 words
```

Stage 3 — Sentence-Based Chunking

A chunk contains a fixed number of consecutive sentences. Overlap allows neighbouring chunks to share sentences, reducing the risk that information at a boundary becomes difficult to retrieve.

```
def split_sentences(text):
    text = re.sub(r"\s+", " ", text.strip())
    sentences = re.split(r"(?<=[.!?])\s+", text)
    return [s.strip() for s in sentences if s.strip()]

def chunk_document(text, source, chunk_size=3, overlap=1):
    if overlap >= chunk_size:
        raise ValueError("overlap must be smaller than chunk_size")
    sentences = split_sentences(text)
    chunks = []
    step = chunk_size - overlap
    for number, start in enumerate(range(0, len(sentences), step)):
        end = min(start + chunk_size, len(sentences))
        chunks.append({
            "chunk_id": f"{source}_chunk_{number}",
            "source": source,
            "start_sentence": start,
            "end_sentence": end - 1,
            "text": " ".join(sentences[start:end])
        })
```

```

    })
    if end == len(sentences):
        break
    return chunks

def build_chunks(documents, chunk_size=3, overlap=1):
    return [c for source, text in documents.items() for c in
            chunk_document(text, source, chunk_size, overlap)]

chunks = build_chunks(documents, 3, 1)
chunks_df = pd.DataFrame(chunks)
print("Number of chunks indexed:", len(chunks))
display(chunks_df[["chunk_id", "source", "start_sentence", "end_sentence",
                    "text"]])

```

Number of chunks indexed: 22

```

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"chunk_id","rawType":"str","type":"string"},
{"name":"source","rawType":"str","type":"string"},
{"name":"start_sentence","rawType":"int64","type":"integer"},
{"name":"end_sentence","rawType":"int64","type":"integer"},
{"name":"text","rawType":"str","type":"string"}],"ref":"9723c88b-4beb-496f-a0e7-e2df0e61267e","rows":
[["0","RAG_Guide.txt_chunk_0","RAG_Guide.txt","0","2","Retrieval-Augmented Generation, or RAG, combines information retrieval with text generation. A RAG system retrieves relevant documents or chunks before generating an answer. The retrieved context is supplied to the language model as evidence."],
["1","RAG_Guide.txt_chunk_1","RAG_Guide.txt","2","4","The retrieved context is supplied to the language model as evidence. RAG can reduce hallucination by grounding answers in an external document collection. The external collection can be updated without retraining the language model."],
["2","RAG_Guide.txt_chunk_2","RAG_Guide.txt","4","6","The external collection can be updated without retraining the language model. TF-IDF is a traditional technique for representing text as numerical vectors. TF-IDF gives importance to words that occur frequently in a document but less frequently across the complete corpus."],
["3","RAG_Guide.txt_chunk_3","RAG_Guide.txt","6","8","TF-IDF gives importance to words that occur frequently in a document but less frequently across the complete corpus. Stop words can be removed during TF-IDF vectorisation. Cosine similarity measures the angle between two vectors."],
["4","RAG_Guide.txt_chunk_4","RAG_Guide.txt","8","10","Cosine similarity measures the angle between two vectors. A higher cosine similarity means that two vectors are more similar. In this experiment cosine similarity is used to rank document chunks for a query."],
["5","RAG_Guide.txt_chunk_5","RAG_Guide.txt","10","12","In this experiment cosine similarity is used to rank document chunks for a query. Chunking divides large documents into smaller pieces."],

```

Overlapping chunks allow information near a chunk boundary to appear in more than one chunk."],
["6","RAG_Guide.txt_chunk_6","RAG_Guide.txt","12","14","Overlapping chunks allow information near a chunk boundary to appear in more than one chunk. Chunk size and overlap affect what information can be retrieved. A vector store contains vector representations of documents or chunks."],["7","RAG_Guide.txt_chunk_7","RAG_Guide.txt","14","16","A vector store contains vector representations of documents or chunks. At query time, the query is converted into a vector and compared against stored vectors. This experiment performs exhaustive similarity search over the complete chunk matrix."],
["8","RAG_Guide.txt_chunk_8","RAG_Guide.txt","16","18","This experiment performs exhaustive similarity search over the complete chunk matrix. A similarity threshold can be used to prevent the system from answering when no retrieved chunk is sufficiently similar to the query. Without a threshold, top-k retrieval always returns some chunks, even when the corpus does not contain the requested information."],
["9","RAG_Guide.txt_chunk_9","RAG_Guide.txt","18","20","Without a threshold, top-k retrieval always returns some chunks, even when the corpus does not contain the requested information. Citations make retrieved evidence traceable. Each chunk keeps its source document and position so that a generated answer can identify where its evidence came from."],
["10","RAG_Guide.txt_chunk_10","RAG_Guide.txt","20","22","Each chunk keeps its source document and position so that a generated answer can identify where its evidence came from. Dense sentence embeddings can replace TF-IDF vectors in a more advanced RAG system. A FAISS index can also replace exhaustive search for large collections."],
["11","RAG_Guide.txt_chunk_11","RAG_Guide.txt","22","23","A FAISS index can also replace exhaustive search for large collections. The chunking and citation logic can remain unchanged."],
["12","AI_Notes.txt_chunk_0","AI_Notes.txt","0","2","Artificial intelligence systems can use external knowledge bases to improve factual responses. A knowledge base may contain technical documents, manuals, policies, or frequently updated information. Information retrieval ranks pieces of information according to their relevance to a user question."],
["13","AI_Notes.txt_chunk_1","AI_Notes.txt","2","4","Information retrieval ranks pieces of information according to their relevance to a user question. A retrieval system should return evidence that directly supports an answer. A grounded answer should be based only on the supplied evidence."],
["14","AI_Notes.txt_chunk_2","AI_Notes.txt","4","6","A grounded answer should be based only on the supplied evidence. An ungrounded language model may provide fluent information without identifying its source. Therefore, citations improve the traceability and auditability of generated answers."],
["15","AI_Notes.txt_chunk_3","AI_Notes.txt","6","8","Therefore,

```

citations improve the traceability and auditability of generated
answers. Retrieval quality depends on document preprocessing. Poor
chunk boundaries can separate related information and make retrieval
less effective."],
["16","AI_Notes.txt_chunk_4","AI_Notes.txt","8","10","Poor chunk
boundaries can separate related information and make retrieval less
effective. Using overlapping chunks can reduce this problem.
Increasing k causes more chunks to be retrieved."],
["17","AI_Notes.txt_chunk_5","AI_Notes.txt","10","11","Increasing k
causes more chunks to be retrieved. This can increase the amount of
evidence available to the generator, but irrelevant chunks may also
enter the context."],
["18","ML_Notes.txt_chunk_0","ML_Notes.txt","0","2","Machine learning
models learn patterns from data. Classification predicts a category
for an input while regression predicts a numerical value. Vector
representations are commonly used in machine learning."],
["19","ML_Notes.txt_chunk_1","ML_Notes.txt","2","4","Vector
representations are commonly used in machine learning. Text can be
represented using sparse vectors such as TF-IDF or dense vectors
produced by neural networks. Similarity search is useful when a query
needs to find related examples."],
["20","ML_Notes.txt_chunk_2","ML_Notes.txt","4","6","Similarity search
is useful when a query needs to find related examples. Cosine
similarity is commonly used for comparing text vectors. A retrieval
system should be evaluated separately from the generation system."],
["21","ML_Notes.txt_chunk_3","ML_Notes.txt","6","7","A retrieval
system should be evaluated separately from the generation system.
Useful retrieval measurements include similarity scores, ranking, and
whether the retrieved chunk contains the required
evidence."]], "shape":{"columns":5,"rows":22}}

```

Stage 4 — TF-IDF Vectorisation

All chunks are vectorised together so they share a common vocabulary. Stop words are removed. The resulting matrix is the simple in-memory vector store.

```

vectorizer = TfidfVectorizer(stop_words="english")
chunk_matrix = vectorizer.fit_transform([c["text"] for c in chunks])
print("TF-IDF matrix shape:", chunk_matrix.shape)
print("Number of indexed chunks:", chunk_matrix.shape[0])
print("Vocabulary size:", len(vectorizer.vocabulary_))

```

```

TF-IDF matrix shape: (22, 186)
Number of indexed chunks: 22
Vocabulary size: 186

```

Stage 5 — Retrieval Function

The query is transformed into a TF-IDF vector. Cosine similarity is calculated against every stored chunk, and the top-k chunks are returned in descending similarity order.

```
def retrieve(query, vectorizer, chunk_matrix, chunks, k=3):
    qv = vectorizer.transform([query])
    scores = cosine_similarity(qv, chunk_matrix).flatten()
    indices = np.argsort(scores)[::-1][:k]
    return [{"rank":r, "chunk_id":chunks[i]["chunk_id"],
"source":chunks[i]["source"], "score":float(scores[i]),
"text":chunks[i]["text"]} for r,i in enumerate(indices,1)]

for item in retrieve("What is cosine similarity?", vectorizer,
chunk_matrix, chunks, 3):
    print(f"Rank {item['rank']} | Score {item['score']:.4f} |
{item['source']} | {item['chunk_id']}")
    print(item["text"]); print("-"*80)
```

Rank 1 | Score 0.6897 | RAG_Guide.txt | RAG_Guide.txt_chunk_4
Cosine similarity measures the angle between two vectors. A higher cosine similarity means that two vectors are more similar. In this experiment cosine similarity is used to rank document chunks for a query.

Rank 2 | Score 0.3864 | ML_Notes.txt | ML_Notes.txt_chunk_2
Similarity search is useful when a query needs to find related examples. Cosine similarity is commonly used for comparing text vectors. A retrieval system should be evaluated separately from the generation system.

Rank 3 | Score 0.2464 | RAG_Guide.txt | RAG_Guide.txt_chunk_5
In this experiment cosine similarity is used to rank document chunks for a query. Chunking divides large documents into smaller pieces. Overlapping chunks allow information near a chunk boundary to appear in more than one chunk.

Stage 6 — Context Assembly

The retrieved chunks are assembled into a grounded prompt. The instruction tells a future language model to use only the supplied context and cite its source.

```
def assemble_grounded_prompt(query, retrieved_chunks):
    context = "\n\n".join(f"[{x['source']} | {x['chunk_id']}] \n{x['text']}" for x in retrieved_chunks)
```

```
return f"""You are a grounded question-answering system.
```

```
Use ONLY the supplied context to answer the question.  
Do not introduce facts that are not supported by the context.  
Include the source identifier of the evidence used.
```

```
CONTEXT:  
{context}
```

```
QUESTION:  
{query}
```

```
ANSWER: ""
```

```
print(assemble_grounded_prompt("What is cosine similarity?",  
retrieve("What is cosine similarity?", vectorizer, chunk_matrix,  
chunks, 3)))
```

You are a grounded question-answering system.

Use ONLY the supplied context to answer the question.
Do not introduce facts that are not supported by the context.
Include the source identifier of the evidence used.

CONTEXT:
[RAG_Guide.txt | RAG_Guide.txt_chunk_4]
Cosine similarity measures the angle between two vectors. A higher cosine similarity means that two vectors are more similar. In this experiment cosine similarity is used to rank document chunks for a query.

[ML_Notes.txt | ML_Notes.txt_chunk_2]
Similarity search is useful when a query needs to find related examples. Cosine similarity is commonly used for comparing text vectors. A retrieval system should be evaluated separately from the generation system.

[RAG_Guide.txt | RAG_Guide.txt_chunk_5]
In this experiment cosine similarity is used to rank document chunks for a query. Chunking divides large documents into smaller pieces. Overlapping chunks allow information near a chunk boundary to appear in more than one chunk.

QUESTION:
What is cosine similarity?

ANSWER:

Stage 7 — Scripted Generation

No external API is used. The top retrieved chunk is treated as evidence for a scripted grounded answer.

```
def grounded_answer(query, retrieved_chunks):
    if not retrieved_chunks:
        return "The answer is not available in the supplied corpus."
    b = retrieved_chunks[0]
    return f"{b['text']} [Source: {b['source']} | {b['chunk_id']}]"

def ungrounded_baseline(query):
    answers = {
        "What is cosine similarity?": "Cosine similarity measures the angle between two vectors and is commonly used to measure their similarity.",
        "What is TF-IDF?": "TF-IDF is a numerical representation that gives importance to terms based on their frequency within documents and the corpus.",
        "What is chunking?": "Chunking divides a large document into smaller pieces that can be processed or retrieved independently.",
        "What is a vector store?": "A vector store contains vector representations that can be searched to find information similar to a query."
    }
    return answers.get(query, "The baseline model would answer using information contained in its learned parameters.")
```

Stage 8 — Complete RAG Pipeline

For every query we print retrieved chunks, similarity scores, grounded answer with citation, and ungrounded baseline answer.

```
queries = [
    "What is cosine similarity?",
    "What is TF-IDF?",
    "What is chunking?",
    "What is a vector store?",
    "How does a similarity threshold help?"
]

for query in queries:
    print("\n" + "=" * 100)
    print("QUERY:", query)
    print("=" * 100)

    results = retrieve(query, vectorizer, chunk_matrix, chunks, 3)
    print("\nRETRIEVED CHUNKS:")
    for x in results:
```

```
print(f"Rank {x['rank']} | Score={x['score']:.4f} |  
{x['source']} | {x['chunk_id']}")  
print(x["text"])
```

```
print("\nGROUNDED ANSWER:")  
print(grounded_answer(query, results))
```

```
print("\nUNGROUNDED BASELINE:")  
print(ungrounded_baseline(query))
```

```
=====
```

```
=====
```

```
QUERY: What is cosine similarity?
```

```
=====
```

```
=====
```

RETRIEVED CHUNKS:

Rank 1 | Score=0.6897 | RAG_Guide.txt | RAG_Guide.txt_chunk_4
Cosine similarity measures the angle between two vectors. A higher cosine similarity means that two vectors are more similar. In this experiment cosine similarity is used to rank document chunks for a query.

Rank 2 | Score=0.3864 | ML_Notes.txt | ML_Notes.txt_chunk_2
Similarity search is useful when a query needs to find related examples. Cosine similarity is commonly used for comparing text vectors. A retrieval system should be evaluated separately from the generation system.

Rank 3 | Score=0.2464 | RAG_Guide.txt | RAG_Guide.txt_chunk_5
In this experiment cosine similarity is used to rank document chunks for a query. Chunking divides large documents into smaller pieces. Overlapping chunks allow information near a chunk boundary to appear in more than one chunk.

GROUNDED ANSWER:

Cosine similarity measures the angle between two vectors. A higher cosine similarity means that two vectors are more similar. In this experiment cosine similarity is used to rank document chunks for a query. [Source: RAG_Guide.txt | RAG_Guide.txt_chunk_4]

UNGROUNDED BASELINE:

Cosine similarity measures the angle between two vectors and is commonly used to measure their similarity.

```
=====
```

```
=====
```

```
QUERY: What is TF-IDF?
```

```
=====
```

```
=====
```

RETRIEVED CHUNKS:

Rank 1 | Score=0.4704 | RAG_Guide.txt | RAG_Guide.txt_chunk_3
TF-IDF gives importance to words that occur frequently in a document but less frequently across the complete corpus. Stop words can be removed during TF-IDF vectorisation. Cosine similarity measures the angle between two vectors.

Rank 2 | Score=0.4647 | RAG_Guide.txt | RAG_Guide.txt_chunk_2
The external collection can be updated without retraining the language model. TF-IDF is a traditional technique for representing text as numerical vectors. TF-IDF gives importance to words that occur frequently in a document but less frequently across the complete corpus.

Rank 3 | Score=0.2507 | ML_Notes.txt | ML_Notes.txt_chunk_1
Vector representations are commonly used in machine learning. Text can be represented using sparse vectors such as TF-IDF or dense vectors produced by neural networks. Similarity search is useful when a query needs to find related examples.

GROUNDING ANSWER:

TF-IDF gives importance to words that occur frequently in a document but less frequently across the complete corpus. Stop words can be removed during TF-IDF vectorisation. Cosine similarity measures the angle between two vectors. [Source: RAG_Guide.txt | RAG_Guide.txt_chunk_3]

UNGROUNDING BASELINE:

TF-IDF is a numerical representation that gives importance to terms based on their frequency within documents and the corpus.

=====

=====

QUERY: What is chunking?

=====

=====

RETRIEVED CHUNKS:

Rank 1 | Score=0.2866 | RAG_Guide.txt | RAG_Guide.txt_chunk_11
A FAISS index can also replace exhaustive search for large collections. The chunking and citation logic can remain unchanged.

Rank 2 | Score=0.2350 | RAG_Guide.txt | RAG_Guide.txt_chunk_5
In this experiment cosine similarity is used to rank document chunks for a query. Chunking divides large documents into smaller pieces. Overlapping chunks allow information near a chunk boundary to appear in more than one chunk.

Rank 3 | Score=0.0000 | ML_Notes.txt | ML_Notes.txt_chunk_1
Vector representations are commonly used in machine learning. Text can be represented using sparse vectors such as TF-IDF or dense vectors produced by neural networks. Similarity search is useful when a query needs to find related examples.

GROUNDING ANSWER:

A FAISS index can also replace exhaustive search for large collections. The chunking and citation logic can remain unchanged. [Source: RAG_Guide.txt | RAG_Guide.txt_chunk_11]

UNGROUNDING BASELINE:

Chunking divides a large document into smaller pieces that can be processed or retrieved independently.

=====

=====

QUERY: What is a vector store?

=====

=====

RETRIEVED CHUNKS:

Rank 1 | Score=0.4958 | RAG_Guide.txt | RAG_Guide.txt_chunk_7
A vector store contains vector representations of documents or chunks. At query time, the query is converted into a vector and compared against stored vectors. This experiment performs exhaustive similarity search over the complete chunk matrix.

Rank 2 | Score=0.3999 | RAG_Guide.txt | RAG_Guide.txt_chunk_6
Overlapping chunks allow information near a chunk boundary to appear in more than one chunk. Chunk size and overlap affect what information can be retrieved. A vector store contains vector representations of documents or chunks.

Rank 3 | Score=0.1134 | ML_Notes.txt | ML_Notes.txt_chunk_1
Vector representations are commonly used in machine learning. Text can be represented using sparse vectors such as TF-IDF or dense vectors produced by neural networks. Similarity search is useful when a query needs to find related examples.

GROUNDING ANSWER:

A vector store contains vector representations of documents or chunks. At query time, the query is converted into a vector and compared against stored vectors. This experiment performs exhaustive similarity search over the complete chunk matrix. [Source: RAG_Guide.txt | RAG_Guide.txt_chunk_7]

UNGROUNDING BASELINE:

A vector store contains vector representations that can be searched to find information similar to a query.

=====

=====

QUERY: How does a similarity threshold help?

=====

=====

RETRIEVED CHUNKS:

Rank 1 | Score=0.5066 | RAG_Guide.txt | RAG_Guide.txt_chunk_8
This experiment performs exhaustive similarity search over the complete chunk matrix. A similarity threshold can be used to prevent the system from answering when no retrieved chunk is sufficiently similar to the query. Without a threshold, top-k retrieval always returns some chunks, even when the corpus does not contain the requested information.

Rank 2 | Score=0.3005 | RAG_Guide.txt | RAG_Guide.txt_chunk_9
Without a threshold, top-k retrieval always returns some chunks, even when the corpus does not contain the requested information. Citations make retrieved evidence traceable. Each chunk keeps its source document and position so that a generated answer can identify where its evidence came from.

Rank 3 | Score=0.1727 | RAG_Guide.txt | RAG_Guide.txt_chunk_4
Cosine similarity measures the angle between two vectors. A higher cosine similarity means that two vectors are more similar. In this experiment cosine similarity is used to rank document chunks for a query.

GROUNDING ANSWER:

This experiment performs exhaustive similarity search over the complete chunk matrix. A similarity threshold can be used to prevent the system from answering when no retrieved chunk is sufficiently similar to the query. Without a threshold, top-k retrieval always returns some chunks, even when the corpus does not contain the requested information. [Source: RAG_Guide.txt | RAG_Guide.txt_chunk_8]

UNGROUNDING BASELINE:

The baseline model would answer using information contained in its learned parameters.

Stage 9 — Effect of Increasing k

Increasing k retrieves more chunks. This can provide more evidence, but it can also introduce irrelevant context.

```
query = "How does chunking improve retrieval?"
for k in [1,2,3,5]:
    print("\n" + "="*80); print(f"k = {k}"); print("="*80)
    for x in retrieve(query, vectorizer, chunk_matrix, chunks, k):
        print(f"Rank {x['rank']} | Score {x['score']:.4f} | {x['source']} | {x['chunk_id']}")
```

```
=====
=====
k = 1
=====
=====
```

```

Rank 1 | Score 0.2100 | AI_Notes.txt | AI_Notes.txt_chunk_3
=====
=====
k = 2
=====
=====
Rank 1 | Score 0.2100 | AI_Notes.txt | AI_Notes.txt_chunk_3
Rank 2 | Score 0.1782 | RAG_Guide.txt | RAG_Guide.txt_chunk_9
=====
=====
k = 3
=====
=====
Rank 1 | Score 0.2100 | AI_Notes.txt | AI_Notes.txt_chunk_3
Rank 2 | Score 0.1782 | RAG_Guide.txt | RAG_Guide.txt_chunk_9
Rank 3 | Score 0.1606 | RAG_Guide.txt | RAG_Guide.txt_chunk_11
=====
=====
k = 5
=====
=====
Rank 1 | Score 0.2100 | AI_Notes.txt | AI_Notes.txt_chunk_3
Rank 2 | Score 0.1782 | RAG_Guide.txt | RAG_Guide.txt_chunk_9
Rank 3 | Score 0.1606 | RAG_Guide.txt | RAG_Guide.txt_chunk_11
Rank 4 | Score 0.1576 | RAG_Guide.txt | RAG_Guide.txt_chunk_8
Rank 5 | Score 0.1317 | RAG_Guide.txt | RAG_Guide.txt_chunk_5

```

Exercise 1 — Similarity Threshold

Top-k retrieval always returns chunks even when the corpus cannot answer the question. A similarity threshold makes the system abstain when the best score is below the threshold. We test two answerable and two unanswerable questions.

```

def retrieve_with_threshold(query, vectorizer, chunk_matrix, chunks,
k=3, threshold=0.20):
    results = retrieve(query, vectorizer, chunk_matrix, chunks, k)
    return {"answerable": bool(results) and results[0]["score"] >=
threshold, "results": results}

threshold = 0.20
tests = [("What is cosine similarity?", True), ("What is TF-IDF?", True),
("What is the capital of France?", False), ("Who invented the
telephone?", False)]
rows=[]

```

```

for q, expected in tests:

r=retrieve_with_threshold(q,vectorizer,chunk_matrix,chunks,3,threshold
)
    pred=r["answerable"]; best=r["results"][0]["score"] if
r["results"] else 0
    rows.append({"Question":q,"Expected":"Answerable" if expected else
"Not in corpus","Best similarity":round(best,4),"Decision":"ANSWER" if
pred else "ABSTAIN","Correct":pred==expected})
display(pd.DataFrame(rows))

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"Question","rawType":"str","type":"string"},
{"name":"Expected","rawType":"str","type":"string"},{"name":"Best
similarity","rawType":"float64","type":"float"},
{"name":"Decision","rawType":"str","type":"string"},
{"name":"Correct","rawType":"bool","type":"boolean"}],"ref":"c4e99d3c-
6951-4275-809d-9394f4f48e9d","rows":[["0","What is cosine
similarity?","Answerable","0.6897","ANSWER","True"],["1","What is TF-
IDF?","Answerable","0.4704","ANSWER","True"],["2","What is the capital
of France?","Not in corpus","0.0","ABSTAIN","True"],["3","Who invented
the telephone?","Not in corpus","0.0","ABSTAIN","True"]],"shape":
{"columns":5,"rows":4}}

```

Exercise 1 — Detailed Results

A threshold is a trade-off: a low value accepts more questions but can allow unsupported questions through, while a high value can reject legitimate questions.

```

for q, expected in tests:

r=retrieve_with_threshold(q,vectorizer,chunk_matrix,chunks,3,threshold
)
    print("\n"+"="*90); print("QUESTION:",q)
    if r["answerable"]: print("DECISION: ANSWER\
n",grounded_answer(q,r["results"]))
    else: print("DECISION: ABSTAIN\nAnswer: The answer is not in the
corpus.")
    print("Best similarity:",f"{r['results'][0]['score']:.4f}")

```

```

=====
=====
QUESTION: What is cosine similarity?
DECISION: ANSWER
Cosine similarity measures the angle between two vectors. A higher
cosine similarity means that two vectors are more similar. In this
experiment cosine similarity is used to rank document chunks for a
query. [Source: RAG_Guide.txt | RAG_Guide.txt_chunk_4]

```

Best similarity: 0.6897

```
=====
=====
QUESTION: What is TF-IDF?
DECISION: ANSWER
TF-IDF gives importance to words that occur frequently in a document
but less frequently across the complete corpus. Stop words can be
removed during TF-IDF vectorisation. Cosine similarity measures the
angle between two vectors. [Source: RAG_Guide.txt |
RAG_Guide.txt_chunk_3]
Best similarity: 0.4704
```

```
=====
=====
QUESTION: What is the capital of France?
DECISION: ABSTAIN
Answer: The answer is not in the corpus.
Best similarity: 0.0000
```

```
=====
=====
QUESTION: Who invented the telephone?
DECISION: ABSTAIN
Answer: The answer is not in the corpus.
Best similarity: 0.0000
```

Exercise 1 — Threshold Sensitivity

Different thresholds show the effect of conservative versus permissive retrieval.

```
vals=[0.05,0.10,0.15,0.20,0.25,0.30,0.40]
out=[]
for t in vals:

correct=sum(retrieve_with_threshold(q,vectorizer,chunk_matrix,chunks,3
,t)["answerable"]==expected for q,expected in tests)
    out.append({"Threshold":t,"Correct out of 4":correct})
display(pd.DataFrame(out))

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"Threshold","rawType":"float64","type":"float"},
{"name":"Correct out of
4","rawType":"int64","type":"integer"}],"ref":"19fffe42-968a-4532-
92f5-aa5e52ffae78","rows":[["0","0.05","4"],["1","0.1","4"],
["2","0.15","4"],["3","0.2","4"],["4","0.25","4"],["5","0.3","4"],
["6","0.4","4"]],"shape":{"columns":2,"rows":7}}
```


Exercise 1 — Observation

Higher similarity generally indicates stronger lexical evidence. A threshold prevents weakly related chunks from automatically becoming evidence, but TF-IDF thresholds can still fail when an unsupported question shares words with the corpus. Threshold selection is therefore an empirical trade-off.

Exercise 2 — Effect of Chunk Size and Overlap

We compare four settings: **(2,0)**, **(3,1)**, **(4,1)**, and **(5,2)** where each pair is (chunk size, overlap) in sentences. For fixed questions, we report the rank and similarity score of the answer-containing chunk.

```
fixed=["What is cosine similarity?", "What is TF-IDF?", "What is chunking?", "What is a vector store?", "Why are citations useful?"]
settings=[(2,0), (3,1), (4,1), (5,2)]
keywords={"What is cosine similarity?": "cosine similarity", "What is TF-IDF?": "TF-IDF", "What is chunking?": "Chunking", "What is a vector store?": "vector store", "Why are citations useful?": "Citations"}
rows=[]
for size, overlap in settings:
    cs=build_chunks(documents, size, overlap)
    vz=TfidfVectorizer(stop_words="english");
    mx=vz.fit_transform([c["text"] for c in cs])
    for q in fixed:
        results=retrieve(q, vz, mx, cs, len(cs))
        found=next((x for x in results if keywords[q].lower() in x["text"].lower()), None)
        rows.append({"Chunk Size": size, "Overlap": overlap, "Question": q, "Answer Chunk": found["chunk_id"] if found else None, "Rank": found["rank"] if found else None, "Similarity": found["score"] if found else None})
exercise2_df=pd.DataFrame(rows)
display(exercise2_df)
```

```
{"columns": [{"name": "index", "rawType": "int64", "type": "integer"}, {"name": "Chunk Size", "rawType": "int64", "type": "integer"}, {"name": "Overlap", "rawType": "int64", "type": "integer"}, {"name": "Question", "rawType": "str", "type": "string"}, {"name": "Answer Chunk", "rawType": "str", "type": "string"}, {"name": "Rank", "rawType": "int64", "type": "integer"}, {"name": "Similarity", "rawType": "float64", "type": "float"}], "ref": "542d48ff-4fcd-49f8-9bd9-389ade9e5e7e", "rows": [{"0", "2", "0", "What is cosine similarity?", "RAG_Guide.txt_chunk_4", "1", "0.6386223137536325"}, {"1", "2", "0", "What is TF-IDF?", "RAG_Guide.txt_chunk_3", "1", "0.4707291387609775"}, {"2", "2", "0", "What is chunking?", "RAG_Guide.txt_chunk_5", "1", "0.2839889844015302"},
```

```
[
  ["3", "2", "0", "What is a vector store?", "RAG_Guide.txt_chunk_7", "1", "0.6062659708321976"],
  ["4", "2", "0", "Why are citations useful?", "AI_Notes.txt_chunk_3", "1", "0.20815532320213193"],
  ["5", "3", "1", "What is cosine similarity?", "RAG_Guide.txt_chunk_4", "1", "0.6896683429963004"],
  ["6", "3", "1", "What is TF-IDF?", "RAG_Guide.txt_chunk_3", "1", "0.4703790966197273"],
  ["7", "3", "1", "What is chunking?", "RAG_Guide.txt_chunk_11", "1", "0.28664424327863863"],
  ["8", "3", "1", "What is a vector store?", "RAG_Guide.txt_chunk_7", "1", "0.4958292718626073"],
  ["9", "3", "1", "Why are citations useful?", "AI_Notes.txt_chunk_2", "3", "0.15751424821829638"],
  ["10", "4", "1", "What is cosine similarity?", "RAG_Guide.txt_chunk_3", "1", "0.44346505919040236"],
  ["11", "4", "1", "What is TF-IDF?", "RAG_Guide.txt_chunk_2", "1", "0.34234956572098985"],
  ["12", "4", "1", "What is chunking?", "RAG_Guide.txt_chunk_7", "1", "0.2102370598822545"],
  ["13", "4", "1", "What is a vector store?", "RAG_Guide.txt_chunk_4", "1", "0.43505591234075713"],
  ["14", "4", "1", "Why are citations useful?", "AI_Notes.txt_chunk_2", "3", "0.12565812682739239"],
  ["15", "5", "2", "What is cosine similarity?", "RAG_Guide.txt_chunk_2", "1", "0.5265907670717742"],
  ["16", "5", "2", "What is TF-IDF?", "RAG_Guide.txt_chunk_1", "1", "0.4287625327635743"],
  ["17", "5", "2", "What is chunking?", "RAG_Guide.txt_chunk_7", "1", "0.21963502416970745"],
  ["18", "5", "2", "What is a vector store?", "RAG_Guide.txt_chunk_4", "1", "0.40207746311486753"],
  ["19", "5", "2", "Why are citations useful?", "AI_Notes.txt_chunk_2", "3", "0.10255189627003401"]], "shape": {"columns": 6, "rows": 20}}
```

Exercise 2 — Compact Comparison

Lower rank and higher similarity are generally preferable. The summary compares retrieval behaviour across the four chunking configurations.

```
summary=(exercise2_df.groupby(["Chunk Size", "Overlap"]).agg(Average_Rank=("Rank", "mean"), Average_Similarity=
("Similarity", "mean"), Questions_Found=("Rank", lambda x:x.notna().sum()), Total_Questions=("Rank", "count")).reset_index())
display(summary)

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"Chunk Size","rawType":"int64","type":"integer"},
```

```
{
  "name": "Overlap", "rawType": "int64", "type": "integer",
  "name": "Average_Rank", "rawType": "float64", "type": "float",
  "name": "Average_Similarity", "rawType": "float64", "type": "float",
  "name": "Questions_Found", "rawType": "int64", "type": "integer",
  "name": "Total_Questions", "rawType": "int64", "type": "integer"}], "ref": "af61d5b1-1204-4b6b-85bb-daafb1811449", "rows":
[[["0", "2", "0", "1.0", "0.4415523461900939", "5", "5"],
["1", "3", "1", "1.4", "0.42000704059511407", "5", "5"],
["2", "4", "1", "1.4", "0.31135314479235926", "5", "5"],
["3", "5", "2", "1.4", "0.33592353667799146", "5", "5"]], "shape":
{"columns": 6, "rows": 4}}
```

Exercise 2 — Select a Configuration

The following cell ranks configurations by the number of answer-containing chunks found, then average similarity, then average rank.

```
ranking=summary.sort_values(["Questions_Found", "Average_Similarity", "Average_Rank"], ascending=[False, False, True])
display(ranking)
best=ranking.iloc[0]
print(f"Recommended configuration: chunk size={int(best['Chunk Size'])}, overlap={int(best['Overlap'])}")
print(f"Questions found: {int(best['Questions_Found'])}/{int(best['Total_Questions'])}")
print(f"Average similarity: {best['Average_Similarity']:.4f}")
print(f"Average rank: {best['Average_Rank']:.2f}")

{"columns": [{"name": "index", "rawType": "int64", "type": "integer"},
{"name": "Chunk Size", "rawType": "int64", "type": "integer"},
{"name": "Overlap", "rawType": "int64", "type": "integer"},
{"name": "Average_Rank", "rawType": "float64", "type": "float"},
{"name": "Average_Similarity", "rawType": "float64", "type": "float"},
{"name": "Questions_Found", "rawType": "int64", "type": "integer"},
{"name": "Total_Questions", "rawType": "int64", "type": "integer"}], "ref": "e11037f0-b659-4093-9495-855bc22d44de", "rows":
[[["0", "2", "0", "1.0", "0.4415523461900939", "5", "5"],
["1", "3", "1", "1.4", "0.42000704059511407", "5", "5"],
["3", "5", "2", "1.4", "0.33592353667799146", "5", "5"],
["2", "4", "1", "1.4", "0.31135314479235926", "5", "5"]], "shape":
{"columns": 6, "rows": 4}}
```

```
Recommended configuration: chunk size=2, overlap=0
Questions found: 5/5
Average similarity: 0.4416
Average rank: 1.00
```

Exercise 2 — Observation

Chunk size and overlap directly affect retrieval. Small chunks are precise but can separate related information; larger chunks preserve context but may introduce irrelevant text. Overlap reduces the risk of losing information at chunk boundaries.

Final Experiment Summary

Main Observations

1. TF-IDF provides a simple and interpretable representation of document chunks.
2. Cosine similarity ranks chunks according to query similarity.
3. Source identifiers make retrieved evidence traceable.
4. Grounded answers are tied to retrieved evidence, unlike the ungrounded baseline.
5. Increasing `k` provides more context but may introduce irrelevant chunks.
6. A similarity threshold enables abstention when retrieval confidence is low.
7. Chunk size and overlap affect retrievability and ranking.
8. Dense embeddings and FAISS could replace TF-IDF and exhaustive search in a production system.

Conclusion

The experiment demonstrates a complete retrieval-augmented pipeline without an external API. The corpus is chunked, vectorised, searched, and supplied as evidence. Citations improve traceability, while thresholding and chunking experiments expose practical RAG limitations.